SISTEMA FCT · GUÍA DE IMPLEMENTACIÓN Y USO

Webhooks locales

Cómo se reciben y envían eventos entre Sistema FCT y otro servicio, y cómo probar el flujo completo en tu equipo sin sustituir la aplicación existente.

> **Idea principal:** el servidor que ya usas en el puerto 8000 queda intacto. La instancia habilitada para webhooks escucha solo en tu equipo, en el puerto 8001.

Proyecto: Sistema FCT (PHP, XAMPP y MariaDB)

Modo descrito: desarrollo y prueba local en Windows

Fecha de preparación: octubre de 2026

Documento complementario: `docs/webhooks.md`

Contenido

1. Qué se implementó y qué no cambia
2. Cómo viaja un evento por la aplicación
3. Preparar secretos y arrancar la instancia local
4. Probar el envío y la recepción, paso a paso
5. Formato, firma y protección contra duplicados
6. Eventos de negocio y manejo de la cola
7. Verificación, diagnóstico y seguridad
8. Archivos principales y límites de esta versión

1. Qué se implementó

Un webhook es una notificación HTTP que un sistema envía a otro cuando ocurre un hecho. La integración añade comunicación en los dos sentidos:

- **Entrada:** un emisor envía JSON firmado a `POST /webhook.php`. Sistema FCT valida la solicitud y conserva el evento en la bandeja de entrada.
- **Salida:** cuando se crea o revisa una oferta, convenio o práctica, Sistema FCT encola una notificación. Un proceso separado la envía al receptor configurado.

> **La recepción archiva, no ejecuta.** Los eventos entrantes no crean ni modifican alumnos, empresas, cursos, ofertas, convenios, prácticas ni calificaciones. Para importar datos haría falta definir otro contrato y sus reglas de permisos y validación.

La cola de salida se escribe en la misma transacción que la acción de negocio. Si esa acción se revierte, no queda una notificación huérfana. El envío HTTP se hace después y no bloquea el formulario de usuario.

> **Se conservan las funciones existentes.** La prueba añade el puerto 8001 y no reemplaza Apache/XAMPP ni el servidor que ya escucha en 8000. Ambos sirven el proyecto y consultan la base local configurada.

2. Cómo viaja un evento



1. Una acción de negocio produce un evento JSON con identificador único.
2. La aplicación inserta el evento en `webhook_outbox` como parte de la transacción de negocio.
3. El worker reclama eventos pendientes, los firma y llama por HTTP a la URL configurada.
4. El endpoint receptor comprueba cabeceras, hora, firma y estructura JSON.
5. Si es válido, conserva el sobre firmado en `webhook_inbox`. El contenido no se interpreta como una instrucción.

3. Preparación local en Windows

Requisitos ya comprobados

- XAMPP/PHP disponible en `C:\xampp\php\php.exe`.
- MariaDB accesible en `127.0.0.1` y base `sistema_fct`.
- Extensión PHP PDO_MYSQL y cURL habilitadas.
- Tablas de webhook creadas mediante la migración 007.
- Puerto 8001 libre al preparar esta instancia.

La configuración local se guarda en `.env.local`. Ese archivo está excluido de Git para evitar subir secretos por accidente. El lanzador solo admite variables de conexión a la base y configuración de webhooks; no imprime los valores sensibles.

Crear el secreto local

Abre PowerShell en la carpeta del proyecto y genera 32 bytes aleatorios. No uses una contraseña de usuario:

```
$rng = [System.Security.Cryptography.RandomNumberGenerator]::Create()
$bytes = New-Object byte[] 32
$rng.GetBytes($bytes)
$secret = [Convert]::ToBase64String($bytes)
$rng.Dispose()
```

Guarda el mismo secreto para entrada y salida porque en esta prueba el worker entrega al endpoint de la propia aplicación:

```
$secret = 'PEGA_AQUI_EL_SECRETO_GENERADO'
@"
FCT_DB_HOST=127.0.0.1
FCT_DB_NAME=sistema_fct
FCT_WEBHOOK_INBOUND_SECRET=$secret
FCT_WEBHOOK_OUTBOUND_SECRET=$secret
FCT_WEBHOOK_OUTBOUND_URL=http://127.0.0.1:8001/webhook.php
"@ | Set-Content -LiteralPath .env.local -Encoding ascii
```

> **No pongas el valor real del secreto en documentación, capturas, correo ni Git.** No guardes ese secreto en archivos compartidos. En la configuración real local ya se generó un valor aleatorio; este documento no lo reproduce.

Arrancar el receptor local

En la primera ventana de PowerShell, desde la raíz del proyecto, ejecuta:

```
Set-Location C:\xampp\htdocs\Practicas_Desarrollo
powershell.exe -NoProfile -ExecutionPolicy Bypass -File scripts\local_webhooks.ps1 -Mode Server
```

El servidor escucha únicamente en `127.0.0.1:8001`. Mantén abierta esta ventana. No detengas ni reemplaces el servidor habitual de Apache o el de desarrollo que utiliza el puerto 8000.

Abre `http://127.0.0.1:8001` para usar la aplicación en la instancia local preparada para webhooks.

4. Prueba completa, paso a paso

1. Confirma que MariaDB está iniciado en XAMPP.
2. Arranca el servidor local en la primera ventana con el modo `Server` descrito arriba.
3. Entra en `http://127.0.0.1:8001` y usa las funciones normales de la aplicación para crear o revisar una oferta, convenio o práctica. La operación elegida encola su evento de salida.
4. Abre una segunda ventana PowerShell en la raíz del proyecto y procesa la cola:

   ```
   Set-Location C:\xampp\htdocs\Practicas_Desarrollo
   powershell.exe -NoProfile -ExecutionPolicy Bypass -File scripts\local_webhooks.ps1 -Mode
   Dispatch
   ```

5. Busca el mensaje `Entregado <event_id>` y el resumen con `"entregados":1` o más.
6. Repite el comando `Dispatch` cuando quieras procesar los siguientes eventos. El worker no queda ejecutándose como servicio en segundo plano.

> El cambio de negocio se produce en la base local. Usa datos de prueba cuando sea posible y evita crear ofertas, convenios o asignaciones de prueba en expedientes reales. El documento no ejecuta acciones en la aplicación ni inserta datos de negocio.

Consulta de estado sin mostrar el JSON completo

Para ver las notificaciones recientes en MariaDB, consulta solo identificadores, tipo y estado:

```
& 'C:\xampp\mysql\bin\mysql.exe' -h 127.0.0.1 -uroot --database=sistema_fct `
  -e "SELECT event_id,event_type,received_at FROM webhook_inbox ORDER BY received_at DESC LIMIT 10;"

& 'C:\xampp\mysql\bin\mysql.exe' -h 127.0.0.1 -uroot --database=sistema_fct `
  -e "SELECT event_id,event_type,attempts,http_status,delivered_at,failed_at,last_error FROM
webhook_outbox ORDER BY created_at DESC LIMIT 10;"
```

No uses `SELECT payload` en capturas o registros compartidos: el contenido es el JSON íntegro recibido o enviado.

5. Formato y seguridad del evento

Sobre JSON

Los dos sentidos usan un sobre versionado, con un identificador irrepetible y un objeto `data`:

```
{
  "event_id": "sistema-externo-evt-001",
  "event": "curso.matricula_actualizada",
  "version": 1,
  "occurred_at": "2026-10-15T10:00:00Z",
  "data": {
    "referencia_externa": "matricula-902"
  }
}
```

Firma HMAC

El emisor añade tres cabeceras. La firma se calcula sobre el cuerpo exacto en bytes; si se vuelve a serializar el JSON, cambia la firma:

```
X-FCT-Event-Id: sistema-externo-evt-001
X-FCT-Timestamp: 1792058400
X-FCT-Signature: sha256=<HMAC hexadecimal>

HMAC-SHA256(secreto, timestamp + "." + cuerpo_JSON_exacto)
```

- La hora solo puede diferir hasta cinco minutos del reloj del servidor.
- El ID de la cabecera debe coincidir con el ID del JSON.
- El cuerpo no puede superar 64 KiB y debe ser JSON válido.
- El endpoint requiere POST y `Content-Type: application/json`.
- La firma de salida se valida con un secreto de al menos 32 caracteres.

Respuestas habituales del receptor

HTTP	Significado
202	Evento válido guardado por primera vez.
200	Evento idéntico ya recibido; repetición idempotente.
400	JSON o estructura del evento no válida.
401	Firma, fecha o cabeceras no válidas.
409	Se reutilizó un event_id con otro contenido.
413	El cuerpo supera el límite.
415	El tipo de contenido no es JSON.
503	Falta el secreto de entrada en la instancia.

6. Eventos de salida y reintentos

Evento	Cuándo	Datos operativos
`convenio.creado`	Se registra un convenio	IDs, especialidad, estado y periodo
`convenio.revisado`	Se aprueba o rechaza	ID y estado
`oferta.creada`	Se registra una oferta	IDs, estado, plazas, fechas y turno
`oferta.revisada`	Se aprueba o rechaza	ID y estado
`practica.asignada`	Se confirma una asignación	IDs de asignación, oferta y alumno
`practica.cerrada`	Se cierra una práctica	ID de asignación

Los datos salientes evitan nombres, correos, DNI, Seguridad Social, direcciones, datos de tutores y documentos. Los identificadores internos siguen siendo información que debe protegerse y solo compartirse con un receptor autorizado.

El worker procesa hasta 25 eventos por ejecución (ajustable de 1 a 100). Una respuesta 2xx marca el evento como entregado. Errores de red y ciertos errores temporales se reintentan con espera exponencial hasta 12 intentos. Los errores permanentes 3xx/4xx (salvo 408, 425 y 429) requieren intervención. Una respuesta superior a 16 KiB se rechaza.

> La entrega puede repetirse si el receptor guardó el evento pero se perdió su respuesta. El receptor externo debe deduplicar por `event_id`. La cola no promete entrega exactamente una vez.

7. Diagnóstico y uso seguro

Síntoma	Qué comprobar
`Address already in use`	El puerto 8001 está ocupado. No detengas el puerto 8000; identifica el proceso de 8001 antes de decidir si liberar ese puerto.
El lanzador dice que falta `.env.local`	Comprueba que el archivo está en la raíz del proyecto, no dentro de `scripts`.
Secreto demasiado corto o distinto	Los dos valores deben tener 32 caracteres o más y ser iguales para el loopback de prueba.
El worker informa cero procesados	No hay eventos vencidos pendientes. Genera una acción compatible desde la instancia 8001 y vuelve a despachar.
Error de conexión a MariaDB	Inicia MariaDB y revisa `FCT_DB_HOST`, `FCT_DB_NAME` y las credenciales locales.
`401` o `400` al enviar	Confirma que servidor y worker leyeron el mismo archivo local, que los relojes están sincronizados y que el payload conserva JSON válido.
`HTTP 4xx` permanente	Inspecciona el código y error guardados en outbox; corrige el receptor antes de reprogramar manualmente el evento.

Finalizar la sesión

Detén el servidor adicional con **Ctrl+C** en la primera ventana. El worker termina al finalizar el comando. El servidor habitual sigue funcionando; el archivo `.env.local` permanece para futuras sesiones y no es necesario compartirlo.

Buenas prácticas de seguridad y privacidad

- El servidor local escucha en loopback. No lo expongas a la red ni a Internet.
- En producción, usa HTTPS y secretos distintos, rotables y guardados fuera del repositorio.
- La bandeja de entrada conserva el cuerpo JSON íntegro; acuerda campos permitidos y una política de retención antes de usar datos reales.
- No transmitas datos de menores, categorías especiales o documentos sin necesidad, autorización y base legal aprobadas.
- El worker no sigue redirecciones, valida TLS para destinos HTTPS y solo permite HTTP para loopback de desarrollo.
- El modo local no cambia los controles existentes de carga y análisis antivirus. Su configuración sigue siendo independiente.

8. Archivos y límites de esta versión

Archivo	Responsabilidad
`app/Core/Webhook.php`	Construye sobres, firma/verifica, valida y deduplica.
`public/webhook.php`	Endpoint HTTP de recepción JSON.
`scripts/dispatch_webhooks.php`	Worker CLI: reclama, firma, envía y reintenta.
`scripts/local_webhooks.ps1`	Lee la configuración local ignorada y arranca el servidor o el worker.
`database/migrations/007_webhooks.sql`	Crea `webhook_inbox` y `webhook_outbox`.
`docs/webhooks.md`	Contrato técnico y referencia de configuración.
`.env.local`	Secretos locales; no debe subirse a Git.

Esta versión no incluye una pantalla para administrar destinos o reintentos, una API REST para consultar expedientes, ni importación automática de entidades académicas. El envío saliente requiere ejecutar el worker manualmente o programarlo; localmente se ejecuta cuando se solicita con el modo `Dispatch`.

> **Resultado esperado:** una acción compatible crea una fila en outbox; el worker la envía al endpoint local; el endpoint responde 202 y la guarda en inbox. La prueba de extremo a extremo se verificó en la base local y se eliminaron las filas sintéticas usadas para probarla.
